Failure (a loop that won't converge, and a cost-cutting pass) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `convergence_and_cost_cutting_practice.py`: two production risks in one file: a revision loop that never agrees, and a pipeline that costs too much. Both are proven here with a hard limit and a measured cut, before Project 4 depends on either. **If not:** Project 4's Writer/Reviewer pair could loop without a clear report, and a "cheaper" prompt could quietly break answers.

The exercise has two parts, both in one file: a generator↔critic loop that never converges (and reports *why*), and a cost-cutting pass that proves a cut is safe by comparing before and after.

Solution

```
# convergence_and_cost_cutting_practice.py
from typing import TypedDict

from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

MAX_REVISIONS = 3


# ---- Part 1: a loop that won't converge, and a useful "why" report ----
class LoopState(TypedDict):
    draft: str
    revision_count: int
    converged: bool
    rejection_history: list


def writer(state: LoopState) -> Command:
    draft = f"[draft attempt {state['revision_count'] + 1}]"
    return Command(update={"draft": draft}, goto="critic")


def critic(state: LoopState) -> Command:
    # impossible criteria on purpose: nothing here could ever pass
```

```
    passed = False
    reason = "draft must be under 10 words and cite 3 sources; neither met"
    history = state["rejection_history"] + [reason]

    if passed:
        return Command(update={"converged": True}, goto=END)

    new_count = state["revision_count"] + 1
    if new_count >= MAX_REVISIONS:
        return Command(
            update={
                "revision_count": new_count,
                "converged": False,
                "rejection_history": history,
            },
            goto=END,
        )
    return Command(
        update={"revision_count": new_count, "rejection_history": history},
        goto="writer",
    )


def run_loop() -> None:
    builder = StateGraph(LoopState)
    builder.add_node("writer", writer)
    builder.add_node("critic", critic)
    builder.add_edge(START, "writer")
    graph = builder.compile()

    start = {
        "draft": "",
        "revision_count": 0,
        "converged": False,
        "rejection_history": [],
    }
    result = graph.invoke(start)
    if result["converged"]:
        print("Accepted:", result["draft"])
        return
    attempts = result["revision_count"]
    print(f"Couldn't converge after {attempts} attempts. Reasons:")
    for reason in result["rejection_history"]:
        print(f"  - {reason}")


# ---- Part 2: a cost-cutting pass, proven safe ----
```

```
# Each stage is (name, tokens). A real run measures tokens from
# response.usage; here they're fixed so the comparison is reproducible.
def total_tokens(stages: list) -> int:
    total = 0
    for name, tokens in stages:
        total = total + tokens
    return total


def cut_most_expensive(stages: list) -> list:
    most = 0
    for index in range(len(stages)):
        if stages[index][1] > stages[most][1]:
            most = index
    cut = []
    for index in range(len(stages)):
        name, tokens = stages[index]
        if index == most:
            tokens = tokens // 2    # a shorter prompt on the priciest stage
        cut.append((name, tokens))
    return cut


def compare_cost() -> None:
    before = [("plan", 200), ("research", 1200), ("write", 400)]
    after = cut_most_expensive(before)
    before_cost = total_tokens(before)
    after_cost = total_tokens(after)
    saved = (before_cost - after_cost) * 100 // before_cost
    print(f"before: {before_cost} tokens, after: {after_cost} tokens")
    print(f"saved {saved}% by halving the 'research' prompt")


if __name__ == "__main__":
    run_loop()
    print()
    compare_cost()
```

Expected output:

```
Couldn't converge after 3 attempts. Reasons:
  - draft must be under 10 words and cite 3 sources; neither met
  - draft must be under 10 words and cite 3 sources; neither met
  - draft must be under 10 words and cite 3 sources; neither met
```

```
before: 1800 tokens, after: 1200 tokens
saved 33% by halving the 'research' prompt
```